АГЕНТЫ ИИ / РАЗРАБОТКА ИИ / ИНФРАСТРУКТУРА ИИ

Чип Хайен объясняет, как сократить расходы на логический вывод без использования нового оборудования

В преддверии ее возвращения на конференцию P99 CONF 21–22 октября советы Чипа Хайена о том, как сократить расходы на ИИ, приобретают особую актуальность в эпоху агентов.

13 сентября 2026 года, 11:00, автор Тим Купманс



Фото Пускаса Адриана на Unsplash

ScyllaDB.

В октябре прошлого года на конференции P99 — онлайн-мероприятии для разработчиков, ориентированных на высокопроизводительные приложения с низкой задержкой, — выступил с потрясающим докладом Чип Хайен.

вот на логический вывод приходится тратить одни и те же средства снова и снова. Это хорошо для поставщиков передовых моделей, но плохо для нас, тех, кто сжигает токены. По подсчетам Хьюена, в течение жизненного цикла модели соотношение затрат на обучение и логический вывод составляет от 1:10 до 1:100. Модели рассуждений, которые сжигают еще больше токенов, еще больше увеличивают это соотношение. Всем нам знакомо чувство, когда мы не успеваем выполнить недельную норму сессий.

По мнению Хайен, если вывод данных обходится слишком дорого, то никто никогда не окупит затраты на обучение, что может объяснить, почему так много мемов о «рентабельности» передовых моделей. Так как же оптимизировать вывод данных?

Этой теме Хюен посвятила несколько месяцев исследований для своей книги. А в духе оптимизации Хюен сократила свой доклад до 30 минут для конференции в октябре 2025 года.

Через несколько недель Хюен вернется на P99 CONF 2026. А пока посмотрите ее полное выступление — или прочитайте краткий обзор ниже — за прошлый год, а затем давайте поговорим о том, как изменились ее идеи за последние 11 месяцев.

ScyllaDB

ScyllaDB разработана для обеспечения предсказуемой производительности при масштабировании. Ее выбирают организации, которым нужна сверхнизкая задержка даже при миллионах операций в секунду и петабайтах данных. Наша уникальная архитектура использует возможности современной инфраструктуры, что позволяет сократить количество узлов, объем администрирования и расходы.

Узнать больше →

ПОСЛЕДНИЕ НОВОСТИ ОТ SCYLLADB

Внутренние механизмы баз данных. Доклады на конференции P99 CONF 2026
12 октября 2026 года

Создание более быстрого драйвера C# для ScyllaDB... на нашей собственной платформе C#-Rust FFI

22 сентября 2026 года

УЗНАЙТЕ БОЛЬШЕ ОТ НАШЕГО СПОНСОРА

Введите свой адрес электронной почты

Отправить

YOUTUBE.COM/THENEWSTACK

Технологии развиваются стремительно, не пропустите ни одного выпуска. Подпишитесь на наш канал на YouTube, чтобы смотреть все наши подкасты, интервью, демонстрации и многое другое.

Подписка

Что измерять

Чип рекомендует сосредоточиться на нескольких ключевых показателях задержки:

- Время до появления первого токена (TTFT): сколько времени проходит до того, как пользователь что-то увидит

- Сквозная задержка: время до появления первого токена плюс время до появления каждого выходного токена, умноженное на количество выходных токенов минус один



(*Click to enlarge graphic.*)

With reasoning models, some of those tokens never reach the user. “The first generated token might not be the same as the first visible token,” Huyen explained. “The model might think for a while, and it will only show the first token of the final output to the user.”

TRENDING STORIES

1. You can outgrow vanilla Postgres without abandoning Postgres
2. "Time to clean up human slop": Why AI now reviews code better than your teammate.
3. The AI safety check that runs on a laptop and nearly matched a 35B model
4. Kill the code review theater, keep the review
5. Use Graph RAG when relationships are part of the evidence

token,"
— Chip Huyen

Some people also measure Time to Publish for that (i.e., how long until the user sees the first token). The best metric to prioritize depends on what matters most for your users.

Also consider "goodput" alongside throughput. Throughput measures requests processed in a given window. Goodput measures the requests that actually met your targets. Chip's example: an app targets 200 ms time to first token and 100 ms time per output token, and processes 10 requests per minute, but only three hit both.



(*Click to enlarge graphic.*)

3 ways to optimize LLM inference

With inference servers, you can optimize from 3 different angles: the hardware, the model, and the service that manages the requests and responses.

(*Click to enlarge graphic.*)

Huyen previously worked at Nvidia and opted out of the hardware discussion: “Even though I find it to be an intellectually interesting topic, it’s not relevant to a lot of people because we don’t have the power to change the hardware itself,” Huyen explained. She also didn’t want to spend much time on the obvious solution: replica parallelism, or just adding more machines. It’s costly, and it gets complicated fast – especially if you end up with a mix of 80GB, 48GB and 24GB machines and models of varying sizes to distribute across them.

That leaves the model and the service. Huyen offers these tips on how to decide: “If you want to host the models yourself, or if you have access to the model weights, or if you train a model yourself, or you want to fine-tune or distill a model, then model optimizations might be for you. However, if you want to take a model as-is and make it more efficient on your own inference service, you might want to look into service optimizations.

Model optimization

The following techniques change the actual weights so that they can change the model outputs.

Quantization lowers the precision used to store weights and activations (e.g., from ۴ bytes per parameter at 32-bit to one byte at 8-bit). Huyen explained, “Reducing ۷ precision not only reduces the memory requirement to run the model, making it

eight times.

The tradeoff is a small quality hit. Huyen continued: “It’s possible to reduce a lot of the model’s memory footprint with minimal quality degradation, and quantization is pretty generalizable to a wide variety of model architectures and model sizes. That’s why it’s very popular. I rarely see any companies running a model at full precision anymore.”

> “I rarely see any companies running a model at full precision anymore.”
> — Chip Huyen

Distillation involves using a large model to generate training data for a smaller model. For example, say you have a truly large model (the example Huyen used was o1) and want a model that performs like it, but is much smaller. Basically, you collect a large set of prompts, run them through the larger model, then train the smaller model on its responses.

Proceed with caution, though. Huyen warned, “A lot of model providers have the condition that they do not allow their models to be used to train competitive models. So even though it’s a very common technique, you need to check licensing.”

Service optimization

This set of techniques targets how requests are scheduled, routed, and reused. The actual weights aren’t affected.

Batching groups multiple requests so they’re processed together in a single pass through the model – which is much more efficient than dealing with them one at a time. Huyen presented a few batching options:

- **Static batching** waits for the batch to fill. This maximizes compute utilization, but it might increase the latency for the first requests.
- **Dynamic batching** runs on a timer instead (e.g., batching every 15 ms). This is less compute-efficient, but it’s better for latency.

request's slot sits idle until the slowest one completes – and new requests queue up behind it. Continuous batching returns each request as it finishes and fills the spot with another request. That can improve compute resource utilization and latency.



(*Click to enlarge graphic.*)

Decoupling prefill and decode separates the two phases of a request onto different machines. (Prefill processes the input, while decode generates the output.) Huyen said, "Input tokens can be processed in parallel, whereas output tokens need to be generated sequentially. With parallel processing, it's bounded by compute, the processing power of the chip. With decoding, it's bounded by memory, because you have to move model weights."

Because each phase stresses different resources, most services now separate them. To improve *time to first token*, shift machines toward prefill. If you care more about improving *time per output token*, shift them to decode.

(*Click to enlarge graphic.*)

Parallelism splits work across machines. Replica parallelism copies the whole model onto more machines. Tensor parallelism divides a very large matrix, so different machines compute different parts of it. Pipeline parallelism divides the model by layer, so requests move through as a pipeline.



(*Click to enlarge graphic.*)

Prompt caching processes shared text once, saving cost and latency. A lot of repetition exists across requests to the same application: the system prompt, the examples, the same code base, the same document behind different questions. You might as well process that shared segment once, cache it, and reuse it.

included prompt caching in the book, and I'm very happy to see that nowadays it's pretty much everywhere."



(*Click to enlarge graphic.*)

The savings scale depending on how much of your prompt gets cached. In Claude Code logs, Huyen's open-source tool Sniffly found cache hit rates of 90% to 97%. Some providers rewrite prompts internally to improve hit rates, but you might as well structure them yourself.

Huyen's tip: since caching works on shared prefixes, put the stable parts of your prompt first and the variable parts later. "It's pretty easy to do, and it can improve your application performance significantly," she noted.

Evaluating inference providers

Huyen closed with a warning for anyone evaluating inference providers: "There are many inference companies that provide inference optimizations for models you want to use, and a lot of them advertise just cost and latency.

"But pay attention to how many inference optimization techniques also change the model behavior or reduce the model quality. So when evaluating an inference

benchmarks?

What's changed one year later?

So where do we stand today, one year on from this keynote? Most of it actually aged quite well.

On the economics, I reckon Huyen was bang on… I think, for most of us as users, we don't have all the cost levers to pull that Huyen outlined. But it's great to understand what is happening. As a novice local LLM user myself, I found I could relate to her points on parallelism (I don't have it) and prompt caching/quantization (within my grasp of control).

Prompt caching (which Huyen said was new when Huyen wrote *AI Engineering*) is now priced into every bundle purchase of API tokens. And her Claude Code observation (90% cache hit rates) is probably the reason we mere mortals can still afford agentic coding agents at all.

Some of it aged in ways that were hard to predict at the time. Huyen mentioned how reasoning models make inference even more significant. One year on, I think agents running multi-step loops with tool calls have turned that idea from a footnote into a way to turn Claude's rate limits (and their infamous 99.x% availability) on their head.

All the metrics Huyen described – time to first token, time to publish, goodput under a latency SLO, etc., are all now part of the lingo and probably need to be reasoned about differently.

That's one thing I hope she's talking about this year! Grab a free conference pass and join us online.

Grab a complimentary pass to PG 99 Conf 2026 and join us on October 21 and 22 to chat with Huyen.

TNS



testing platform. After it was acquired, he enjoyed...

Read more from Tim Koopmans →

SHARE THIS STORY

TRENDING STORIES

1. You can outgrow vanilla Postgres without abandoning Postgres
2. "Time to clean up human slop": Why AI now reviews code better than your teammate.
3. The AI safety check that runs on a laptop and nearly matched a 35B model
4. Kill the code review theater, keep the review
5. Use Graph RAG when relationships are part of the evidence

INSIGHTS FROM OUR SPONSOR

ScyllaDB

ScyllaDB is engineered to deliver predictable performance at scale. It's adopted by organizations that need ultra-low latency, even over millions of ops/sec & PBs of data. Our unique architecture leverages the power of modern infrastructure – translating to fewer nodes, less admin & lower costs.
Learn More →

Inside the Database Internals Talks at P99 CONF 2026
October 2026

Building a Faster (Rust-Based) Python Driver for ScyllaDB

C#-Rust FFI Framework

22 September 2026

Bringing QUIC to Seastar

14 September 2026

Full-Text Search, Object Storage Backend, and More in ScyllaDB 2026.3

8 September 2026

A Self-Baked Async FFI Framework for Rust C# Interop

31 August 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

EMAIL ADDRESS

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our Terms of Use and Privacy Policy.

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS

Disclosures
Terms of Use
Advertising Terms & Conditions
Privacy Policy
Cookie Policy